



Kapsamlı Teknik Dokümantasyon

Hazırlayanlar:

MASIHULLAH OMAR (170422921)

ALİ RIZA MISTİK (170423851)

Tarih: 30 Aralık 2025

Icindekiler

1	Giriş	2
1.1	NexOS Nedir?	2
1.2	Yapay Zeka Entegrasyonu	2
1.3	Temel Kavramlar	3
1.3.1	Süreç (Process)	3
1.3.2	Bellek İzolasyonu	3
1.3.3	Zamanlayıcı (Scheduler)	3
1.4	Sistem Gereksinimleri	3
2	Geliştirme Ortamı Kurulumu	4
2.1	Bağımlılıkların Kurulumu	4
2.1.1	Ubuntu/Debian	4
2.1.2	Fedora/RHEL	4
2.1.3	macOS (Sadece Derleme)	4
2.2	Proje Dizin Yapısı	5
2.3	Projeyi Sıfırdan Oluşturma	5
2.3.1	Adım 1: Proje Dizinini Oluştur	5
2.3.2	Adım 2: Makefile Oluştur	5
2.4	globals.h - Merkezi Tanımlamalar	6
2.4.1	Header Guard ve Kütüphane Bildirimleri	6
2.4.2	Sabit Tanımlamalar	6
2.4.3	Enum Tanımlamaları	7
2.4.4	Veri Yapıları	7
2.4.5	Global Değişkenler ve Fonksiyon Prototipleri	8
2.5	main.c - Program Giriş Noktası	9
2.6	Derleme ve Test	10
3	Sistem Mimarisi	11
3.1	Genel Bakış	11
3.2	Kaynak Dosyaları	11
3.3	Derleme ve Çalıştırma	11
3.4	Katmanlı Mimari	12
3.5	Veri Akışı	12
3.6	Modül Etkileşim Matrisi	12
3.7	Merkezi Header Dosyası	13

4	Süreç Yönetimi	14
4.1	Süreç Durumları	14
4.2	Süreç Kontrol Bloğu (PCB)	14
4.3	Surec Olusturma	15
4.4	Surec Sonlandirma	15
5	Bellek Yönetimi	16
5.1	İzole Bellek Modeli	16
5.2	Belleğe Yazma	16
5.3	Bellekten Okuma	17
5.4	Neden Sınır Kontrolü?	17
6	CPU Zamanlayıcısı	18
6.1	Round-Robin Algoritması	18
6.2	Zaman Dilimi	18
6.3	Zamanlayıcı Tick	19
7	Süreçler Arası İletişim (IPC)	20
7.1	Mesaj Geçirme Modeli	20
7.2	Mesaj Gönderme	20
7.3	Mesaj Alma	21
8	Dosya Sistemi	22
8.1	Sanal Disk	22
8.2	Dosya İşlemleri	22
9	Log Analiz Araçları	23
9.1	Genel Bakış	23
9.2	Web Sunucu Log Dosyaları	23
9.3	Modüler Script Yapısı	24
9.3.1	Yardımcı Fonksiyonlar	24
9.4	Trafik Analizi Fonksiyonları	25
9.4.1	En Aktif Ziyaretçileri Bulma	25
9.4.2	En Çok Ziyaret Edilen Sayfalar	25
9.4.3	HTTP Durum Kodları Dağılımı	25
9.4.4	Hata İsteklerini Analiz Etme	26
9.4.5	Saatlik Trafik Analizi	26
9.4.6	Bant Genişliği Hesaplama	27
9.5	Güvenlik Tehdit Tespiti	27
9.6	Rapor Üretimi	28
9.6.1	Metin Raporu	28
9.6.2	HTML Rapor ve Dashboard	29
9.7	Cron ile Otomatik Zamanlama	30
9.7.1	Cron İfade Formatı	31
9.8	Canlı İzleme Modu	31
9.9	Ana Program Akışı	32
9.10	Kullanım Örnekleri	33

10 Yapay Zeka Entegrasyonu	34
10.1 Genel Bakış	34
10.2 Ollama AI Motoru	34
10.3 Özel Model Oluşturma	34
10.4 AI Modülü (ai.c)	35
10.5 Kullanıcıdan Soru Alma	36
10.6 Komut Öneri ve Çalıştırma	36
10.7 AI Modu Kullanım	38
11 Entegre Web Sunucu	39
11.1 Genel Bakış	39
11.2 Web Sunucu Scripti	39
11.3 Dizin Yapısı Kurulumu	40
11.4 Web Sayfası Oluşturma	40
11.5 HTTP Kimlik Doğrulama	41
11.6 Sunucu Başlatma ve Durdurma	42
11.7 Canlı Log Takibi	42
11.8 Ana Program	43
12 ISO Oluşturma ve Boot Süreci	44
12.1 Genel Bakış	44
12.2 Derleme Süreci	44
12.3 Root Dosya Sistemi	44
12.4 Init Script	45
12.5 Initramfs Oluşturma	46
12.6 ISO İmajı Oluşturma	46
12.7 ISO Çalıştırma	47
13 Kullanıcı Arayüzü	48
13.1 Giriş Ekranı	48
13.2 Ana Menü	48
13.3 Shell Komutları	48
14 Sonuç	49
14.1 Özellik Özeti	49
14.2 Öğrenilen Kavramlar	49
15 Ekler	50
15.1 Ekran Görüntüleri	50
15.1.1 Giriş Ekranı	50
15.1.2 Ana Ekran	50
15.1.3 Yardım Sayfası	51
15.1.4 Süreç Yönetimi (proc kill)	51
15.1.5 Sistem İzleme (monitor)	51
15.1.6 Log Analiz Çıktısı	52
15.1.7 Web Sitesi Görünümü	52
15.1.8 Yapay Zeka Yanıtı	52

Bolum 1

Giriş

1.1 NexOS Nedir?

NexOS, modern işletim sistemi kavramlarını gösteren eğitim amaçlı bir sistemdir. Kullanıcılara:

- Süreç yönetimi ve izolasyon
- Bellek tahsisi ve koruma
- CPU zamanlama algoritmaları
- Dosya sistemi işlemleri
- Web sunucu yönetimi ve log analizi
- Yapay zeka destekli komut asistanı

konularında pratik deneyim sağlar.

1.2 Yapay Zeka Entegrasyonu

NexOS, kullanıcılara komut önerilerinde bulunmak için entegre bir yapay zeka asistanı içerir. Bu asistan:

- **Ollama** açık kaynak AI çalışma zamanını kullanır
- **Qwen 1.5B** modelini temel alır (1.5 milyar parametre)
- Özel veri seti üzerinde **fine-tune** edilmiştir
- BusyBox komutlarını önerme konusunda uzmanlaşmıştır
- Tehlikeli komutları filtreleyerek güvenlik sağlar

Fine-tuning süreci, modelin NexOS ortamına özgü komutları daha doğru önermesini sağlar. Model, GGUF formatında paketlenerek ISO içine dahil edilir ve internet bağlantısı gerektirmeden yerel olarak çalışır.

1.3 Temel Kavramlar

1.3.1 Süreç (Process)

Bir **süreç**, çalıştırılan bir programdır. Her süreç:

- Benzersiz bir PID (Process ID) numarasına sahiptir
- Kendi izole bellek alanında çalışır
- Farklı durumlarda (NEW, READY, RUNNING, BLOCKED, TERMINATED) bulunabilir

1.3.2 Bellek İzolasyonu

Her süreç kendi özel bellek alanına sahiptir. Bir süreç, başka bir sürecin belleğine erişemez. Bu güvenlik için kritik öneme sahiptir.

1.3.3 Zamanlayıcı (Scheduler)

CPU aynı anda sadece bir süreç çalıştırır. **Zamanlayıcı**, hangi sürecin ne zaman çalışacağına karar verir. NexOS, **Round-Robin** algoritmasını kullanır.

1.4 Sistem Gereksinimleri

NexOS çalıştırmak için:

- GCC derleyici (statik derleme destekli)
- Make (derleme otomasyonu)
- QEMU veya VirtualBox (ISO test için)
- xorriso (ISO oluşturma)
- wget ve curl (BusyBox/Ollama indirme)
- 2GB RAM ve 5GB disk alanı (önerilen)

Bolum 2

Geliştirme Ortamı Kurulumu

2.1 Bağımlılıkların Kurulumu

2.1.1 Ubuntu/Debian

```
1 # Temel derleme araclari
2 sudo apt update
3 sudo apt install -y build-essential gcc make
4
5 # Statik derleme icin gerekli
6 sudo apt install -y libc6-dev libgcc-s1
7
8 # ISO olusturma araclari
9 sudo apt install -y xorriso isolinux syslinux-common
10
11 # Sanal makine (QEMU)
12 sudo apt install -y qemu-system-x86
13
14 # Indirme araclari
15 sudo apt install -y wget curl
```

Kod 2.1: Gerekli Paketlerin Kurulumu

2.1.2 Fedora/RHEL

```
1 sudo dnf install -y gcc make glibc-static
2 sudo dnf install -y xorriso syslinux
3 sudo dnf install -y qemu-system-x86
```

Kod 2.2: Fedora Kurulumu

2.1.3 macOS (Sadece Derleme)

```
1 # Homebrew ile
2 brew install gcc make qemu
3
4 # NOT: macOS'ta statik derleme ve ISO
5 # olusturma icin Linux VM oneriliyor
```

Kod 2.3: macOS Kurulumu

2.2 Proje Dizin Yapısı

NexOS projesi aşağıdaki dizin yapısına sahiptir:

```

1 nexos/
2 |
3 +-- src/                # C kaynak kodlari
4 |   +-- globals.h       # Ortak tanımlamalar
5 |   +-- main.c          # Ana giriş noktası
6 |   +-- process.c       # Sürec yönetimi
7 |   +-- memory.c        # Bellek yönetimi
8 |   +-- scheduler.c     # CPU zamanlayıcı
9 |   +-- ipc.c           # Süreçler arası iletişim
10 |  +-- fs.c             # Dosya sistemi
11 |  +-- shell.c          # Komut kabuğu
12 |  +-- tui.c            # Terminal arayuzu
13 |  +-- hardware.c       # Donanım bilgileri
14 |  +-- utils.c          # Yardımcı fonksiyonlar
15 |  +-- ai.c             # AI entegrasyonu
16 |
17 +-- scripts/            # Shell scriptleri
18 |   +-- webserver.sh    # Web sunucu
19 |   +-- log_analyzer.sh # Log analizi
20 |   +-- analyze_traffic.sh # Trafik analizi
21 |
22 +-- distro/             # ISO oluşturma
23 |   +-- build_iso.sh    # Ana build scripti
24 |   +-- init            # Boot scripti
25 |
26 +-- Makefile            # Derleme kuralları
27 +-- README.md           # Proje açıklaması

```

Kod 2.4: Proje Yapısı

2.3 Projeyi Sıfırdan Oluşturma

2.3.1 Adım 1: Proje Dizinini Oluştur

```

1 mkdir -p nexos/src nexos/scripts nexos/distro
2 cd nexos

```

2.3.2 Adım 2: Makefile Oluştur

```

1 # Derleyici ayarları
2 CC = gcc
3 CFLAGS = -Wall -Wextra -static -O2
4
5 # Dizinler
6 SRCDIR = src
7 BUILDDIR = build
8
9 # Kaynak dosyaları
10 SOURCES = main.c process.c memory.c scheduler.c \
11           ipc.c fs.c shell.c tui.c hardware.c \

```

```

12         utils.c ai.c
13 OBJECTS = $(SOURCES:.c=.o)
14
15 # Ana hedef
16 all: nexos
17
18 nexos: $(OBJECTS)
19     $(CC) $(CFLAGS) -o $@ $(addprefix $(SRCDIR)/,$^)
20
21 # C dosyalarini derle
22 %.o: $(SRCDIR)/%.c $(SRCDIR)/globals.h
23     $(CC) $(CFLAGS) -c $< -o $(SRCDIR)/$@
24
25 # Temizlik
26 clean:
27     rm -f $(SRCDIR)/*.o nexos
28
29 # ISO olustur
30 iso: nexos
31     cd distro && ./build_iso.sh
32
33 .PHONY: all clean iso

```

Kod 2.5: Makefile

2.4 globals.h - Merkezi Tanımlamalar

Tüm modüller `globals.h` dosyasını include eder. Bu dosya projenin “anayasası” gibidir.

2.4.1 Header Guard ve Kütüphane Bildirimleri

```

1 #ifndef GLOBALS_H
2 #define GLOBALS_H
3
4 // Standart C kutuphaneleri
5 #include <stdio.h>           // printf, scanf, FILE
6 #include <stdlib.h>         // malloc, free, exit
7 #include <string.h>         // strcpy, strcmp, memset
8 #include <unistd.h>         // read, write, access
9 #include <stdbool.h>        // true, false, bool
10 #include <sys/sysinfo.h>    // Sistem bilgileri
11 #include <sys/statvfs.h>    // Disk bilgileri
12 #include <termios.h>        // Terminal kontrolu
13 #include <time.h>           // Zaman fonksiyonlari

```

Kod 2.6: `globals.h` - Baslangic

2.4.2 Sabit Tanımlamalar

```

1 // Versiyon bilgisi
2 #define NEXOS_VERSION "1.2.0"
3
4 // Terminal renk kodlari (ANSI)
5 #define RENK_KIRMIZI "\033[1;31m"

```

```

6 #define RENK_YESIL    "\033[1;32m"
7 #define RENK_MAVI     "\033[1;34m"
8 #define RENK_SARI     "\033[1;33m"
9 #define RENK_CYAN     "\033[1;36m"
10 #define RENK_SIFIRLA  "\033[0m"
11
12 // Dosya sistemi ayarlari
13 #define DISK_DOSYASI  "sanal_disk.bin"
14 #define MAX_DOSYA 50
15 #define MAX_DOSYA_BOYUTU 1024
16
17 // Surec ve bellek ayarlari
18 #define MAX_PROCESS 10
19 #define PROCESS_MEMORY_SIZE 4096 // 4KB
20 #define TIME_QUANTUM 3 // Round-Robin
21 #define MAX_IPC_MESSAGES 10
22 #define MAX_IPC_MSG_SIZE 256

```

Kod 2.7: globals.h - Sabitler

2.4.3 Enum Tanımlamaları

```

1 // Surec durumlari
2 typedef enum {
3     PROC_NEW, // Yeni olusturuldu
4     PROC_READY, // Hazir
5     PROC_RUNNING, // Calisiyor
6     PROC_BLOCKED, // Bloklu
7     PROC_TERMINATED // Sonlandi
8 } ProcessState;

```

Kod 2.8: globals.h - Enumlar

2.4.4 Veri Yapıları

```

1 // IPC Mesaj yapisi
2 typedef struct {
3     int sender_pid;
4     int receiver_pid;
5     char content[MAX_IPC_MSG_SIZE];
6     bool read;
7 } IPCMessage;
8
9 // Surec kontrol blogu (PCB)
10 typedef struct {
11     int pid;
12     char name[32];
13     ProcessState state;
14     int priority;
15     int time_slice;
16
17     // Izole bellek
18     int memory_base;
19     int memory_limit;
20     char memory_space[PROCESS_MEMORY_SIZE];

```

```

21     int memory_used;
22
23     // IPC
24     IPCMessage mailbox[MAX_IPC_MESSAGES];
25     int mailbox_count;
26
27     bool active;
28 } SimProcess;
29
30 // Dosya sistemi blogu
31 typedef struct {
32     char ad[32];
33     int boyut;
34     char icerik[MAX_DOSYA_BOYUTU];
35     bool dolu;
36 } DosyaBlogu;

```

Kod 2.9: globals.h - Struct Tanımlamaları

2.4.5 Global Değişkenler ve Fonksiyon Prototipleri

```

1 // Global tablo (tum moduller erisebilir)
2 extern SimProcess process_table[MAX_PROCESS];
3 extern DosyaBlogu sanal_disk[MAX_DOSYA];
4 extern int pid_counter;
5
6 // === FONKSIYON PROTOTIPLERI ===
7
8 // process.c
9 SimProcess* process_create(char *name, int priority);
10 void process_terminate(int pid);
11 SimProcess* process_find(int pid);
12 void process_list();
13 void process_block(int pid);
14 void process_unblock(int pid);
15
16 // memory.c
17 void memory_init();
18 int mem_write(int pid, int offset, void *data, int size);
19 int mem_read(int pid, int offset, void *buffer, int size);
20 void mem_demo();
21
22 // scheduler.c
23 void scheduler_init();
24 void scheduler_tick();
25 void scheduler_demo();
26
27 // ipc.c
28 int ipc_send(int sender, int receiver, char *msg);
29 int ipc_receive(int pid, char *buffer);
30 void ipc_demo();
31
32 // fs.c
33 void sfs_yukle();
34 void sfs_kaydet();
35 void sfs_dosya_olustur(char *isim, char *icerik);
36 void sfs_listele();

```

```
37
38 // shell.c
39 void shell_run();
40
41 // tui.c
42 int show_login_screen();
43 void tui_menu();
44
45 // ai.c
46 void ai_sor(char *soru);
47 void ai_command_helper_run(char *soru);
48
49 #endif // GLOBALS_H
```

Kod 2.10: globals.h - Extern ve Prototipler

2.5 main.c - Program Giriş Noktası

```
1 #include "globals.h"
2 #include <signal.h>
3
4 // Global degiskenlerin tanimlari
5 SimProcess process_table[MAX_PROCESS];
6 DosyaBlogu sanal_disk[MAX_DOSYA];
7 int pid_counter = 1000;
8
9 // Sinyal handler (temiz cikis)
10 void handle_signal(int sig) {
11     printf("\nNexOS kapatiliyor...\n");
12     sfs_kaydet(); // Dosya sistemini kaydet
13     exit(0);
14 }
15
16 int main() {
17     // Sinyal yakala
18     signal(SIGINT, handle_signal);
19     signal(SIGTERM, handle_signal);
20
21     // Sistemleri baslat
22     sfs_yukle(); // Dosya sistemi
23     memory_init(); // Bellek simulasyonu
24
25     // Giris ekrani
26     if (show_login_screen() != 0) {
27         printf("Giris basarisiz!\n");
28         return 1;
29     }
30
31     // Ana menu
32     tui_menu();
33
34     // Cikista kaydet
35     sfs_kaydet();
36
37     return 0;
38 }
```

Kod 2.11: main.c - Temel Yapi

2.6 Derleme ve Test

```
1 # Dizine git
2 cd nexos
3
4 # Derle
5 make
6
7 # Calistir (normal mod)
8 ./nexos
9
10 # ISO olustur (Linux gerekli)
11 make iso
12
13 # QEMU ile test
14 qemu-system-x86_64 -cdrom distro/nexos.iso -m 2G
```

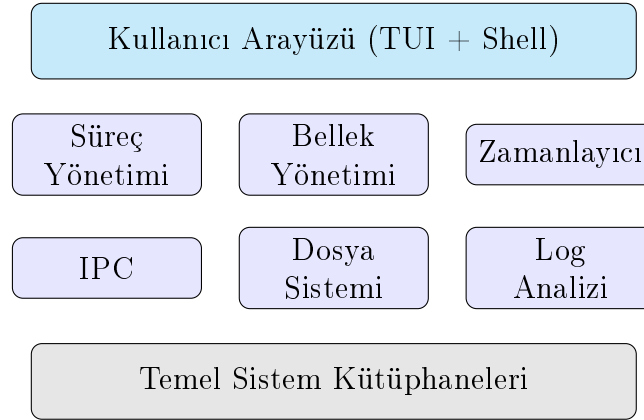
Kod 2.12: Proje Derleme

Bolum 3

Sistem Mimarisi

3.1 Genel Bakış

NexOS modüler bir yapıya sahiptir. Her modül belirli bir görev üstlenir:



Sekil 3.1: NexOS Sistem Mimarisi

3.2 Kaynak Dosyaları

Dosya	Görevi
main.c	Program giriş noktası
globals.h	Global tanımlamalar
process.c	Süreç oluşturma ve yönetimi
memory.c	İzole bellek işlemleri
scheduler.c	Round-Robin zamanlayıcı
ipc.c	Süreçler arası mesajlaşma
fs.c	Sanal dosya sistemi
shell.c	Komut satırı arayüzü
tui.c	Terminal grafik arayüzü
utils.c	Yardımcı fonksiyonlar

3.3 Derleme ve Çalıştırma

```

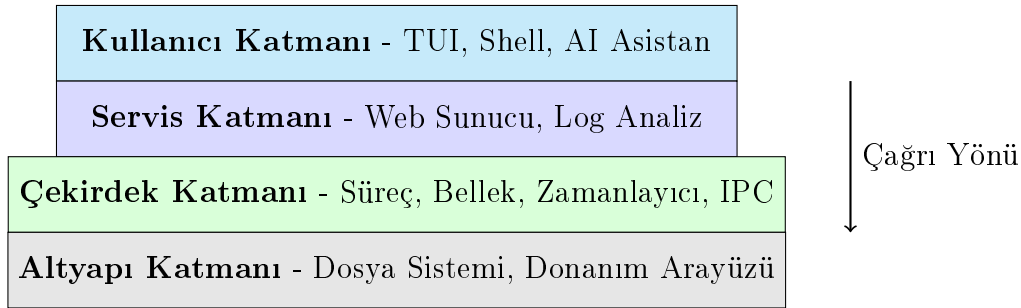
1 # Binary derleme
2 make
3
4 # ISO olusturma
5 make iso
6
7 # QEMU ile calistirma
8 qemu-system-x86_64 -cdrom distro/nexos.iso -m 2G

```

Kod 3.1: Derleme Komutlari

3.4 Katmanlı Mimari

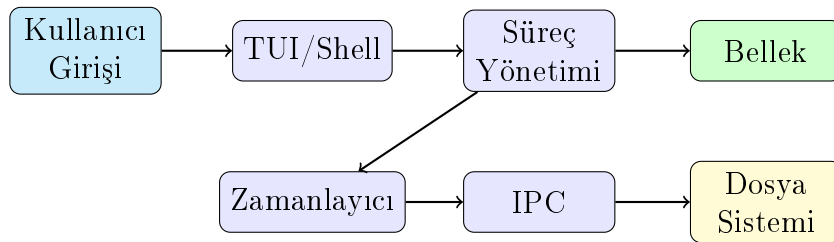
NexOS, katmanlı bir mimari yaklaşımı benimser. Üst katmanlar alt katmanların sunduğu servisleri kullanır:



Sekil 3.2: NexOS Katmanlı Mimari

3.5 Veri Akışı

Sistem içerisindeki temel veri akışı şu şekilde gerçekleşir:



Sekil 3.3: Sistem Veri Akışı

3.6 Modül Etkileşim Matrisi

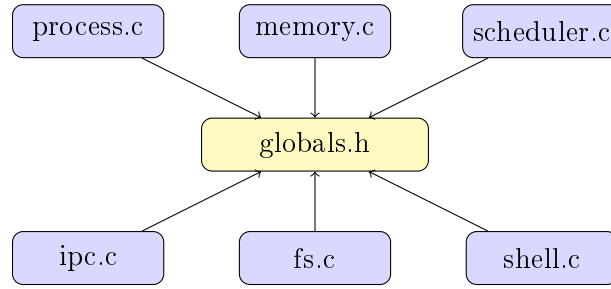
Aşağıdaki tablo hangi modülün hangi modülü kullandığını gösterir:

Modül	process	memory	sched	ipc	fs	shell
shell.c	X	X	X	X	X	-
tui.c	X	-	-	-	-	X
scheduler.c	X	-	-	-	-	-
ipc.c	X	-	-	-	-	-
ai.c	-	-	-	-	-	X

3.7 Merkezi Header Dosyası

Tüm modüller `globals.h` dosyasını dahil eder. Bu dosya:

- Paylaşılan veri yapılarını tanımlar (`SimProcess`, `DosyaBlogu`)
- Sistem sabitlerini belirler (`MAX_PROCESS`, `TIME_QUANTUM`)
- Fonksiyon prototiplerini bildirir (modüller arası çağrılar için)
- ANSI renk kodlarını tanımlar (terminal çıktısı için)



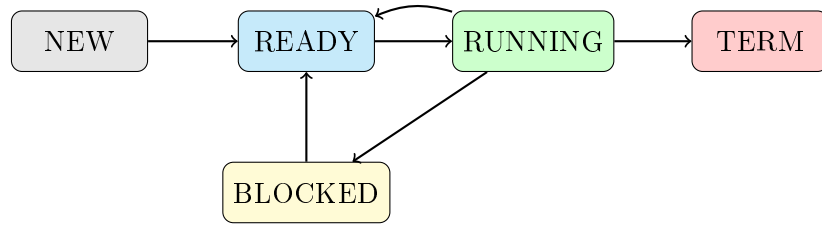
Sekil 3.4: Modül-Header Bağımlılığı

Bolum 4

Süreç Yönetimi

4.1 Süreç Durumları

Bir süreç beş farklı durumda olabilir:



Sekil 4.1: Süreç Durum Geçişleri

- **NEW:** Süreç yeni oluşturuldu
- **READY:** Çalışmaya hazır, CPU bekliyor
- **RUNNING:** CPU üzerinde çalışıyor
- **BLOCKED:** I/O işlemi bekliyor
- **TERMINATED:** Çalışma tamamlandı

4.2 Süreç Kontrol Bloğu (PCB)

Her süreç için tutulan bilgiler:

```
1 typedef struct {
2     int pid; // Benzersiz surec ID
3     char name[32]; // Surec adi
4     ProcessState state; // Mevcut durum
5     int priority; // Oncelik seviyesi (0-9)
6     int time_slice; // Kalan CPU zamani
7
8     // Izole Bellek
9     int memory_base; // Baslangic adresi
10    int memory_limit; // Maksimum boyut (4096 byte)
11    char memory_space[4096]; // Ozel bellek alanı
```

```

12     int memory_used;                // Kullanilan miktar
13
14     // IPC
15     IPCMessage mailbox[10];        // Mesaj kutusu
16     int mailbox_count;             // Bekleyen mesaj sayisi
17
18     bool active;                   // Aktif durumu
19 } SimProcess;

```

Kod 4.1: SimProcess Yapisi

4.3 Surec Olusturma

```

1 SimProcess* process_create(char *name, int priority) {
2     // Bos slot bul
3     for(int i = 0; i < MAX_PROCESS; i++) {
4         if(!process_table[i].active) {
5             SimProcess *proc = &process_table[i];
6
7             // PID ata
8             proc->pid = pid_counter++;
9             strncpy(proc->name, name, 31);
10            proc->state = PROC_READY;
11            proc->priority = priority;
12            proc->time_slice = 3;
13
14            // Izole bellek alani baslat
15            proc->memory_base = i * PROCESS_MEMORY_SIZE;
16            proc->memory_limit = PROCESS_MEMORY_SIZE;
17            memset(proc->memory_space, 0, PROCESS_MEMORY_SIZE);
18
19            proc->active = true;
20            return proc;
21        }
22    }
23    return NULL; // Tablo dolu
24 }

```

Kod 4.2: process_create Fonksiyonu

4.4 Surec Sonlandirma

```

1 void process_terminate(int pid) {
2     SimProcess *proc = process_find(pid);
3     if(!proc) return;
4
5     proc->state = PROC_TERMINATED;
6     proc->active = false;
7
8     // Guvenlik: Bellegi temizle
9     memset(proc->memory_space, 0, PROCESS_MEMORY_SIZE);
10 }

```

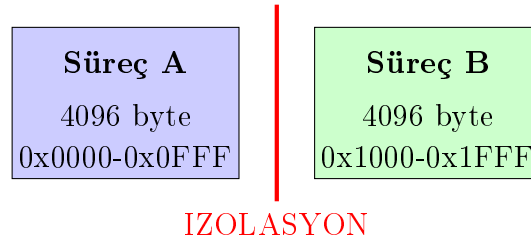
Kod 4.3: process_terminate Fonksiyonu

Bolum 5

Bellek Yönetimi

5.1 İzole Bellek Modeli

Her sürecin 4KB (4096 byte) özel bellek alanı vardır. Süreçler birbirlerinin belleğine erişemez.



Sekil 5.1: Bellek İzolasyonu

5.2 Belleğe Yazma

```
1 int mem_write(int pid, int offset, void *data, int size) {
2     SimProcess *proc = process_find(pid);
3     if(!proc) return -1;
4
5     // Sinir kontrolu
6     if(offset < 0 || offset + size > proc->memory_limit) {
7         printf("SEGMENTATION FAULT!\n");
8         printf("Surec: %s, Offset: %d, Size: %d\n",
9             proc->name, offset, size);
10        proc->state = PROC_TERMINATED;
11        return -1;
12    }
13
14    // Veriyi yaz
15    memcpy(proc->memory_space + offset, data, size);
16
17    if(offset + size > proc->memory_used)
18        proc->memory_used = offset + size;
19
20    return 0;
21 }
```

Kod 5.1: mem_write Fonksiyonu

5.3 Bellekten Okuma

```
1 int mem_read(int pid, int offset, void *buffer, int size) {
2     SimProcess *proc = process_find(pid);
3     if(!proc) return -1;
4
5     // Sınır kontrolü
6     if(offset < 0 || offset + size > proc->memory_limit) {
7         printf("SEGMENTATION FAULT!\n");
8         return -1;
9     }
10
11     memcpy(buffer, proc->memory_space + offset, size);
12     return 0;
13 }
```

Kod 5.2: mem_read Fonksiyonu

5.4 Neden Sınır Kontrolü?

Sınır kontrolü **buffer overflow** saldırılarını önler:

1. Hatalı kod kendi alanının dışına yazamaz
2. Diğer süreçlerin verileri korunur
3. Güvenlik açıkları önlenir

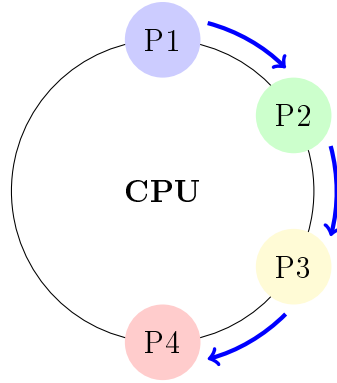
Bolum 6

CPU Zamanlayıcısı

6.1 Round-Robin Algoritması

NexOS, **Round-Robin** zamanlama algoritması kullanır:

1. Her sürece eşit zaman dilimi (quantum) verilir
2. Zaman dolunca süreç kuyruğa alınır
3. Sıradaki süreç çalışmaya başlar
4. Son süreç bitince başa dönülür



Sekil 6.1: Round-Robin Döngüsü

6.2 Zaman Dilimi

```
1 #define TIME_QUANTUM 3 // Her surec 3 tick calisir
```

Kod 6.1: Zaman Dilimi Tanimi

6.3 Zamanlayıcı Tick

```
1 void scheduler_tick() {
2     scheduler_tick_count++;
3
4     // Calisan sureci bul
5     SimProcess *current = NULL;
6     for(int i = 0; i < MAX_PROCESS; i++) {
7         if(process_table[i].state == PROC_RUNNING) {
8             current = &process_table[i];
9             break;
10        }
11    }
12
13    if(current) {
14        current->time_slice--;
15
16        if(current->time_slice <= 0) {
17            // Zaman doldu - preempt
18            current->state = PROC_READY;
19            current->time_slice = TIME_QUANTUM;
20
21            // Siradaki hazir sureci bul
22            int next = find_next_ready();
23            if(next >= 0) {
24                process_table[next].state = PROC_RUNNING;
25            }
26        }
27    }
28 }
```

Kod 6.2: scheduler_tick Fonksiyonu

Bolum 7

Süreçler Arası İletişim (IPC)

7.1 Mesaj Geçirme Modeli

Süreçler izole olduğu için doğrudan iletişim kuramaz. NexOS, **posta kutusu** (mailbox) modeli kullanır.



Sekil 7.1: İstemci-Sunucu Mesajlaşma

7.2 Mesaj Gönderme

```
1 int ipc_send(int sender_pid, int receiver_pid, char *message) {
2     SimProcess *sender = process_find(sender_pid);
3     SimProcess *receiver = process_find(receiver_pid);
4
5     if(!sender || !receiver) return -1;
6
7     // Alicinin posta kutusunda bos yer bul
8     for(int i = 0; i < MAX_IPC_MESSAGES; i++) {
9         if(receiver->mailbox[i].read) {
10             receiver->mailbox[i].sender_pid = sender_pid;
11             strncpy(receiver->mailbox[i].content, message,
12                     MAX_IPC_MSG_SIZE - 1);
13             receiver->mailbox[i].read = false;
14             receiver->mailbox_count++;
15
16             // Alici bloklu ise uyandır
17             if(receiver->state == PROC_BLOCKED) {
18                 receiver->state = PROC_READY;
19             }
20             return 0;
21         }
22     }
23     return -1; // Kutu dolu
```

24 }

Kod 7.1: ipc_send Fonksiyonu

7.3 Mesaj Alma

```
1 int ipc_receive(int pid, char *buffer) {
2     SimProcess *proc = process_find(pid);
3     if(!proc) return -1;
4
5     // İlk okunmamis mesaji bul
6     for(int i = 0; i < MAX_IPC_MESSAGES; i++) {
7         if(!proc->mailbox[i].read) {
8             strcpy(buffer, proc->mailbox[i].content);
9             proc->mailbox[i].read = true;
10            proc->mailbox_count--;
11            return 0;
12        }
13    }
14    return -1; // Mesaj yok
15 }
```

Kod 7.2: ipc_receive Fonksiyonu

Bolum 8

Dosya Sistemi

8.1 Sanal Disk

NexOS, bir sanal disk kullanır. Dosyalar `sanal_disk.bin` dosyasında saklanır.

```
1 typedef struct {
2     char ad[32];           // Dosya adi
3     int boyut;             // Dosya boyutu
4     char icerik[1024];     // Dosya icerigi
5     bool dolu;             // Kullaniliyor mu?
6 } DosyaBlogu;
7
8 DosyaBlogu sanal_disk[50]; // 50 dosya kapasitesi
```

Kod 8.1: Dosya Blogu Yapisi



Sekil 8.1: Sanal Disk Blokları

8.2 Dosya İşlemleri

```
1 void sfs_dosya_olustur(char *isim, char *icerik) {
2     for(int i = 0; i < MAX_DOSYA; i++) {
3         if(!sanal_disk[i].dolu) {
4             strncpy(sanal_disk[i].ad, isim, 31);
5             strncpy(sanal_disk[i].icerik, icerik,
6                 MAX_DOSYA_BOYUTU - 1);
7             sanal_disk[i].boyut = strlen(icerik);
8             sanal_disk[i].dolu = true;
9             sfs_kaydet();
10            return;
11        }
12    }
13 }
```

Kod 8.2: Dosya Olusturma

Bolum 9

Log Analiz Araçları

9.1 Genel Bakış

NexOS, web sunucuların oluşturduğu log dosyalarını analiz etmek için kapsamlı script araçları içerir. Bu araçlar sistem yöneticilerine:

- Ziyaretçi trafiğini anlamak
- Güvenlik tehditlerini tespit etmek
- Performans sorunlarını belirlemek
- Otomatik raporlar oluşturmak

imkanı sağlar. Scriptler shell programlama ile yazılmış olup, modüler ve genişletilebilir bir yapıya sahiptir.

9.2 Web Sunucu Log Dosyaları

Web sunucular her isteği bir log dosyasına kaydeder. Bu dosyalar genellikle “Combined Log Format” formatında tutulur:

```
1 192.168.1.100 - - [30/Dec/2025:10:15:32 +0300] "GET /index.html HTTP/1.1"
   " 200 4523 "-" "Mozilla/5.0"
2 192.168.1.101 - - [30/Dec/2025:10:15:45 +0300] "GET /admin.php HTTP/1.1"
   404 512 "-" "Chrome/90.0"
3 10.0.0.50 - - [30/Dec/2025:10:16:01 +0300] "POST /login HTTP/1.1" 401
   128 "-" "curl/7.68.0"
```

Kod 9.1: access.log Örnek Satırlar

Her satırdaki alanlar:

Alan	Örnek	Anlamı
\$1	192.168.1.100	Ziyaretçi IP adresi
\$4	[30/Dec/2025:10:15:32]	Tarih ve saat
\$6	GET	HTTP metodu
\$7	/index.html	İstenen sayfa
\$9	200	HTTP durum kodu
\$10	4523	Gönderilen byte sayısı

9.3 Modüler Script Yapısı

Log analiz scripti modüler bir yapıda tasarlanmıştır. Her analiz görevi ayrı bir fonksiyon olarak tanımlanır ve ana program bu fonksiyonları çağırarak rapor oluşturur.

```

1 #!/bin/sh
2 # NexOS Log Analyzer - Kapsamli Web Log Analiz Araci
3
4 # Yapilandirma degiskenleri
5 LOG_DIR="/var/log/httpd"
6 ACCESS_LOG="${LOG_DIR}/access.log"
7 REPORT_DIR="/var/www/reports"
8 HTML_REPORT="${REPORT_DIR}/index.html"
9 DATE=$(date +%Y-%m-%d)
10 TIME=$(date +%H:%M:%S)
11
12 # Terminal renkleri
13 RED='\033[0;31m'
14 GREEN='\033[0;32m'
15 YELLOW='\033[1;33m'
16 NC='\033[0m'

```

Kod 9.2: Script Konfigurasyon Bolumu

9.3.1 Yardımcı Fonksiyonlar

Tüm modüller tarafından kullanılan ortak fonksiyonlar:

```

1 # Bilgi mesaji yazdir
2 log_info() {
3     echo -e "${GREEN}[INFO]${NC} $1"
4 }
5
6 # Uyari mesaji yazdir
7 log_warn() {
8     echo -e "${YELLOW}[WARN]${NC} $1"
9 }
10
11 # Hata mesaji yazdir
12 log_error() {
13     echo -e "${RED}[ERROR]${NC} $1"
14 }
15
16 # Gerekli dizinleri olustur
17 ensure_directories() {
18     # Her dizin icin kontrol yap
19     for dir in "$LOG_DIR" "$REPORT_DIR"; do
20         # Dizin yoksa olustur
21         if [ ! -d "$dir" ]; then
22             mkdir -p "$dir"
23             log_info "Dizin olusturuldu: $dir"
24         fi
25     done
26 }

```

Kod 9.3: Yardimci Fonksiyonlar

9.4 Trafik Analizi Fonksiyonları

9.4.1 En Aktif Ziyaretçileri Bulma

Hangi IP adreslerinden en çok istek geldiğini bulmak için:

```

1 get_top_ips() {
2     local log_file="$1"
3     local count="${2:-10}" # Varsayılan: 10
4
5     # Log dosyası var mı kontrol et
6     if [ ! -f "$log_file" ]; then
7         log_error "Log dosyası bulunamadı: $log_file"
8         return 1
9     fi
10
11    # IP'leri cikart, sirala, say ve en coklari goster
12    awk '{print $1}' "$log_file" | \
13        sort | \
14        uniq -c | \
15        sort -rn | \
16        head -n "$count"
17 }

```

Kod 9.4: IP Analizi Fonksiyonu

Bu fonksiyon çalıştığında log dosyasındaki her satırın ilk alanını (IP adresi) alır, bunları sıralar, tekrar edenleri sayar ve en çok tekrar edenleri listeler. Pipe (|) karakteri ile komutlar zincirleme bağlanıyor.

9.4.2 En Çok Ziyaret Edilen Sayfalar

```

1 get_top_pages() {
2     local log_file="$1"
3     local count="${2:-10}"
4
5     # 7. sutun sayfa URL'sini icerir
6     awk '{print $7}' "$log_file" | \
7         grep -v "^-$" | \      # Bos satirlari at
8         sort | \
9         uniq -c | \
10        sort -rn | \
11        head -n "$count"
12 }

```

Kod 9.5: Sayfa Analizi Fonksiyonu

9.4.3 HTTP Durum Kodları Dağılımı

Sunucunun verdiği cevapların dağılımını görmek için:

```

1 get_status_codes() {
2     local log_file="$1"
3
4     # 9. sutun HTTP durum kodunu icerir
5     awk '{print $9}' "$log_file" | \

```

```

6      grep -E "[0-9]{3}$" | \ # Sadece 3 haneli kodlar
7      sort | \
8      uniq -c | \
9      sort -rn
10 }

```

Kod 9.6: Durum Kodu Analizi

Durum kodlarının anlamı:

- **200** - Başarılı istek
- **301/302** - Yönlendirme
- **404** - Sayfa bulunamadı
- **500** - Sunucu hatası

9.4.4 Hata İsteklerini Analiz Etme

404 ve 500 gibi hataları tespit etmek güvenlik ve bakım için önemlidir:

```

1 get_error_requests() {
2     local log_file="$1"
3     local count="${2:-20}"
4
5     # 4xx ve 5xx kodlarini filtrele
6     awk '$9 ~ /^[45][0-9][0-9]$/' {print $9, $7, $1}' "$log_file" | \
7     sort | \
8     uniq -c | \
9     sort -rn | \
10    head -n "$count"
11 }

```

Kod 9.7: Hata Analizi Fonksiyonu

Bu fonksiyon sadece hata kodlarını değil, aynı zamanda hangi sayfaların hangi IP'lerden istek aldıktan sonra hata verdiğini de gösterir.

9.4.5 Saatlik Trafik Analizi

Hangi saatlerde daha yoğun trafik olduğunu anlamak için:

```

1 get_hourly_traffic() {
2     local log_file="$1"
3
4     awk '{
5         # Zaman damgasından saat bilgisini cikar
6         # Format: [DD/Mon/YYYY:HH:MM:SS
7         match($4, /[0-9]{2}/, arr)
8         if (RSTART > 0) {
9             hour = substr($4, RSTART+1, 2)
10            hours[hour]++
11        }
12    }
13    END {
14        # Sonuclari yazdir
15        for (h in hours) {

```

```

16         printf "%s:00 - %d istek\n", h, hours[h]
17     }
18     }, "$log_file" | sort
19 }

```

Kod 9.8: Saatlik Trafik Fonksiyonu

9.4.6 Bant Genişliği Hesaplama

Sunucunun toplam ne kadar veri gönderdiğini hesaplamak için:

```

1 get_bandwidth() {
2     local log_file="$1"
3
4     awk '{
5         total += $10 # 10. sutun byte sayisi
6     }
7     END {
8         # Okunabilir formata cevir
9         if (total > 1073741824) {
10             printf "%.2f GB\n", total/1073741824
11         } else if (total > 1048576) {
12             printf "%.2f MB\n", total/1048576
13         } else if (total > 1024) {
14             printf "%.2f KB\n", total/1024
15         } else {
16             printf "%d bytes\n", total
17         }
18     }, "$log_file"
19 }

```

Kod 9.9: Bant Genisligi Hesaplama

9.5 Güvenlik Tehdit Tespiti

Log dosyalarında bilinen saldırı imzalarını aramak için:

```

1 detect_threats() {
2     local log="$1"
3
4     echo "--- GUVENLIK UYARILARI ---"
5
6     # Directory Traversal, SQL Injection, XSS tespiti
7     if grep -qE "(\\.\\.\\.\\/|etc\\/passwd|union select<script>)" "$log";
8     then
9         echo "[KRITIK] Saldiri girisimi tespit edildi:"
10        grep -E "(\\.\\.\\.\\/|etc\\/passwd|union select<script>)" "$log" |
11        head -n 3
12    else
13        echo "[OK] Bilinen saldiri imzasi bulunamadi."
14    fi
15
16    # Hassas dosyalara erisim tespiti
17    if grep -qE "(admin|login|config|\\.env|\\.git)" "$log"; then
18        echo "[UYARI] Hassas yol erisim girisimi:"
19        grep -E "(admin|login|config)" "$log" | head -n 3
20    fi
21 }

```

```

18     fi
19
20     # Brute Force tespiti (çok sayıda başarısız giriş)
21     local failed_logins=$(grep "401" "$log" | grep "login" | wc -l)
22     if [ "$failed_logins" -gt 5 ]; then
23         echo "[KRITİK] Brute Force saldırısı olabilir!"
24         echo "Başarısız giriş denemesi: $failed_logins"
25     fi
26 }

```

Kod 9.10: Güvenlik Taraması

9.6 Rapor Üretimi

9.6.1 Metin Raporu

Terminal veya e-posta için metin formatında rapor:

```

1 generate_text_report() {
2     local log_file="$1"
3     local output_file="${REPORT_DIR}/rapor_${DATE}.txt"
4
5     log_info "Metin raporu oluşturuluyor..."
6
7     {
8         echo "===== "
9         echo "      NEXOS WEB LOG ANALİZ RAPORU"
10        echo "===== "
11        echo "Tarih: ${DATE} ${TIME}"
12        echo "Log Dosyası: ${log_file}"
13        echo ""
14
15        echo "--- ÖZET ---"
16        echo "Toplam İstek: $(wc -l < "$log_file")"
17        echo "Benzersiz Ziyaretçi: $(awk '{print $1}' "$log_file" | sort
18        -u | wc -l)"
19        echo "Bant Genisliği: $(get_bandwidth "$log_file")"
20        echo ""
21
22        echo "--- EN ÇOK GELEN IP ADRESLERİ ---"
23        get_top_ips "$log_file" 10
24        echo ""
25
26        echo "--- EN ÇOK ZİYARET EDİLEN SAYFALAR ---"
27        get_top_pages "$log_file" 10
28        echo ""
29
30        echo "--- HTTP DURUM KODLARI ---"
31        get_status_codes "$log_file"
32        echo ""
33
34        echo "--- HATALI İSTEKLER ---"
35        get_error_requests "$log_file" 10
36    } > "$output_file"
37
38    log_info "Rapor kaydedildi: $output_file"
39 }

```

Kod 9.11: Metin Rapor Uretimi

9.6.2 HTML Rapor ve Dashboard

Web tarayıcıda görülebilen modern bir dashboard:

```

1 generate_html_report() {
2     local log_file="$1"
3
4     log_info "HTML raporu olusturuluyor..."
5
6     # Istatistikleri hesapla
7     local total_requests=$(wc -l < "$log_file")
8     local unique_visitors=$(awk '{print $1}' "$log_file" | sort -u | wc
9     -l)
10    local bandwidth=$(get_bandwidth "$log_file")
11
12    # HTML dosyasini olustur
13    cat > "$HTML_REPORT" << EOF
14    <!DOCTYPE html>
15    <html>
16    <head>
17        <title>NexOS Log Dashboard</title>
18        <meta http-equiv="refresh" content="30">
19        <style>
20            body { background: #1a1a2e; color: #eee; font-family: sans-serif
21            ; }
22            .card { background: rgba(255,255,255,0.05); padding: 20px;
23            border-radius: 10px; margin: 10px; }
24            .value { font-size: 2em; color: #00ff88; }
25            table { width: 100%; border-collapse: collapse; }
26            th, td { padding: 10px; border-bottom: 1px solid #333; }
27        </style>
28    </head>
29    <body>
30        <h1>NexOS Trafik Analizi</h1>
31
32        <div class="card">
33            <h3>Toplam Istek</h3>
34            <div class="value">${total_requests}</div>
35        </div>
36
37        <div class="card">
38            <h3>Benzersiz Ziyaretci</h3>
39            <div class="value">${unique_visitors}</div>
40        </div>
41
42        <div class="card">
43            <h3>Bant Genisligi</h3>
44            <div class="value">${bandwidth}</div>
45        </div>
46
47        <div class="card">
48            <h2>En Cok Gelen IP Adresleri</h2>
49            <table>
50                <tr><th>Sayi</th><th>IP</th></tr>

```

```

49 EOF
50
51 # IP tablosunu ekle
52 get_top_ips "$log_file" 10 | while read count ip; do
53     echo "<tr><td>${count}</td><td>${ip}</td></tr>" >> "$HTML_REPORT"
54 done
55
56 echo "</table></div></body></html>" >> "$HTML_REPORT"
57
58 log_info "Dashboard kaydedildi: $HTML_REPORT"
59 }

```

Kod 9.12: HTML Dashboard Üretimi

Dashboard her 30 saniyede otomatik yenilenir (meta refresh).

9.7 Cron ile Otomatik Zamanlama

Raporların düzensiz aralıklarda otomatik oluşturulması için cron kullanılır:

```

1 setup_cron() {
2     local interval="${1:-hourly}"
3     local script_path=$(readlink -f "$0")
4     local cron_entry=""
5
6     # Aralığa göre cron ifadesi oluştur
7     case "$interval" in
8         hourly)
9             # Her saat başında çalıştır
10            cron_entry="0 * * * * $script_path --auto"
11            ;;
12        daily)
13            # Her gün gece yarısı çalıştır
14            cron_entry="0 0 * * * $script_path --auto"
15            ;;
16        weekly)
17            # Her pazar gece yarısı çalıştır
18            cron_entry="0 0 * * 0 $script_path --auto"
19            ;;
20        *)
21            log_error "Bilinmeyen aralık: $interval"
22            log_info "Kullanım: hourly, daily, weekly"
23            return 1
24            ;;
25    esac
26
27    # Crontab'a ekle (varsa ekleme)
28    if crontab -l 2>/dev/null | grep -q "$script_path"; then
29        log_warn "Bu script için cron görevi zaten mevcut"
30    else
31        (crontab -l 2>/dev/null; echo "$cron_entry") | crontab -
32        log_info "Cron görevi eklendi: $cron_entry"
33    fi
34 }

```

Kod 9.13: Cron Kurulumu

9.7.1 Cron İfade Formatı

```

1 # Cron formati: dakika saat gun ay haftanin_gunu komut
2 # Ornekler:
3 0 * * * *      # Her saat basinda
4 0 0 * * *      # Her gun gece yarisi
5 0 0 * * 0      # Her pazar gece yarisi
6 */5 * * * *    # Her 5 dakikada bir
7 0 8,18 * * *   # Her gun saat 8 ve 18'de

```

9.8 Canlı İzleme Modu

Sistem yöneticileri log dosyasını canlı olarak izleyebilir:

```

1 live_monitor() {
2     local log_file="$1"
3
4     log_info "Canli izleme baslatiliyor (Ctrl+C ile durdur)..."
5
6     # Sonsuz dongu ile surekli guncelle
7     while true; do
8         clear
9         echo "===== "
10        echo "  NEXOS CANLI LOG MONITOR"
11        echo "===== "
12        echo "Zaman: $(date)"
13        echo "Log: $log_file"
14        echo ""
15
16        echo "--- ANLIK İSTATİSTİKLER ---"
17        echo "Toplam İstek: $(wc -l < "$log_file")"
18        echo "Benzersiz Ziyaretçi: $(awk '{print $1}' "$log_file" | sort
19        -u | wc -l)"
20        echo ""
21
22        echo "--- SON 10 İSTEK ---"
23        tail -10 "$log_file"
24        echo ""
25
26        echo "--- EN AKTİF 5 IP ---"
27        get_top_ips "$log_file" 5
28
29        # 5 saniye bekle ve tekrar et
30        sleep 5
31    done
32 }

```

Kod 9.14: Canlı İzleme Fonksiyonu

Bu mod açık kaldığı sürece ekran her 5 saniyede güncellenir ve sistem yöneticisi anlık trafiği görebilir.

9.9 Ana Program Akışı

Tüm fonksiyonlar tanımlandıktan sonra ana program kullanıcıdan gelen parametrelere göre çalışır:

```
1 main() {
2     # Dizinleri hazırla
3     ensure_directories
4
5     local log_file="$ACCESS_LOG"
6     local action="all"
7
8     # Komut satiri parametrelerini isle
9     while [ $# -gt 0 ]; do
10         case "$1" in
11             --log)
12                 log_file="$2"
13                 shift 2
14                 ;;
15             --text)
16                 action="text"
17                 shift
18                 ;;
19             --html)
20                 action="html"
21                 shift
22                 ;;
23             --setup-cron)
24                 setup_cron "$2"
25                 exit 0
26                 ;;
27             --live)
28                 live_monitor "$log_file"
29                 exit 0
30                 ;;
31             --help)
32                 show_help
33                 exit 0
34                 ;;
35             *)
36                 log_error "Bilinmeyen parametre: $1"
37                 exit 1
38                 ;;
39         esac
40     done
41
42     # Secilen islemi calistir
43     case "$action" in
44         text)
45             generate_text_report "$log_file"
46             ;;
47         html)
48             generate_html_report "$log_file"
49             ;;
50         all)
51             generate_text_report "$log_file"
52             generate_html_report "$log_file"
53             detect_threats "$log_file"
```

```
54         ;;
55     esac
56
57     log_info "Analiz tamamlandi!"
58 }
59
60 # Programi calistir
61 main "$@"
```

Kod 9.15: Ana Program

9.10 Kullanım Örnekleri

```
1 # Tam analiz (metin + HTML + guvenlik)
2 ./log_analyzer.sh
3
4 # Sadece HTML rapor olustur
5 ./log_analyzer.sh --html
6
7 # Farkli bir log dosyasini analiz et
8 ./log_analyzer.sh --log /var/log/nginx/access.log
9
10 # Canli izleme baslat
11 ./log_analyzer.sh --live
12
13 # Gunluk otomatik rapor icin cron kur
14 ./log_analyzer.sh --setup-cron daily
15
16 # Haftalik otomatik rapor icin cron kur
17 ./log_analyzer.sh --setup-cron weekly
18
19 # En cok istekte bulunan 20 IP'yi goster
20 ./log_analyzer.sh --top-ips 20
```

Kod 9.16: Script Kullanım Örnekleri

Bolum 10

Yapay Zeka Entegrasyonu

10.1 Genel Bakış

NexOS, kullanıcılara komut önerilerinde bulunmak ve sistem yönetimini kolaylaştırmak için entegre bir yapay zeka asistanı içerir. Bu asistan:

- Ollama AI çalışma zamanını kullanır
- Özel eğitilmiş “nexos-expert” modelini çalıştırır
- Kullanıcı sorularını analiz ederek BusyBox komutları önerir
- Tehlikeli komutları filtreleyerek güvenlik sağlar

10.2 Ollama AI Motoru

Ollama, yerel olarak çalışan ve çeşitli dil modellerini destekleyen açık kaynak bir AI çalışma zamanıdır. NexOS'ta Ollama sunucu modu ile çalışır:

```
1 # Ollama sunucusunu arka planda baslat
2 /bin/ollama serve > /var/log/ollama.log 2>&1 &
3
4 # Sunucunun hazir olmasini bekle
5 for i in 1 2 3 4 5 6 7 8 9 10; do
6     sleep 2
7     if wget -q -O /dev/null http://127.0.0.1:11434/api/tags; then
8         echo "Ollama hazir!"
9         break
10    fi
11    echo "Bekleniyor... ($i/10)"
12 done
```

Kod 10.1: Ollama Sunucu Baslatma

10.3 Özel Model Oluşturma

NexOS, GGUF formatında bir model dosyası ve Modelfile kullanarak özel bir AI modeli oluşturur:

```

1 # /root/Modelfile
2 FROM /root/nexos-expert.gguf
3
4 PARAMETER temperature 0.1
5 PARAMETER top_p 0.9
6
7 SYSTEM "You are NexOS Assistant, a specialized AI for a custom
8 Linux distro. You ONLY output BusyBox commands in the format:
9 COMMAND: <cmd>
10 EXPLANATION: <text>
11 You are helpful, safe, and concise."

```

Kod 10.2: Modelfile Icerigi

```

1 # Modeli Ollama'ya kaydet
2 /bin/ollama create nexos-expert -f /root/Modelfile
3
4 # Kayitli modelleri listele
5 /bin/ollama list

```

Kod 10.3: Model Kayit Islemi

10.4 AI Modülü (ai.c)

C kodunda AI entegrasyonu şu şekilde yapılır:

```

1 void ensure_ai_model_exists() {
2     if (ai_model_ready) return; // Zaten kontrol edildi
3
4     // Ollama binary var mi?
5     if (access("/bin/ollama", F_OK) != 0) {
6         printf("[AI] Ollama kurulu degil!\n");
7         return;
8     }
9
10    // Modelfile var mi? Yoksa olustur
11    if (access("/root/Modelfile", F_OK) != 0) {
12        printf("[AI] Modelfile bulunamadi, olusturuluyor...\n");
13
14        FILE *f = fopen("/root/Modelfile", "w");
15        if (f) {
16            fprintf(f, "FROM /root/nexos-expert.gguf\n\n");
17            fprintf(f, "PARAMETER temperature 0.1\n");
18            fprintf(f, "PARAMETER top_p 0.9\n\n");
19            fprintf(f, "SYSTEM \"You are NexOS Assistant...\n\n");
20            fclose(f);
21        }
22    }
23
24    // Model kayitli mi kontrol et
25    FILE *check = popen("/bin/ollama list | grep nexos-expert", "r");
26    if (check) {
27        char buf[16] = "";
28        if (fgets(buf, sizeof(buf), check) == NULL) {
29            // Model kayitli degil, kaydet
30            printf("[AI] Model kaydediliyor...\n");

```

```

31         system("/bin/ollama create nexos-expert -f /root/Modelfile")
32     ;
33     }
34     pclose(check);
35 }
36 ai_model_ready = true;
37 }

```

Kod 10.4: AI Model Kontrolü

10.5 Kullanıcıdan Soru Alma

```

1 void ai_sor(char *soru) {
2     char komut[512];
3     char tampon[256];
4
5     ensure_ai_model_exists();
6
7     if (access("/bin/ollama", F_OK) != 0) {
8         printf("[HATA] AI Modulu bulunamadi!\n");
9         return;
10    }
11
12    printf("[AI] NexOS Expert dusuniyor...\n");
13
14    sprintf(komut, "/bin/ollama run nexos-expert \"%s\" 2>&1", soru);
15    FILE *pipe = popen(komut, "r");
16    if (!pipe) {
17        perror("popen");
18        return;
19    }
20
21    printf("--- AI Yaniti ---\n");
22    while (fgets(tampon, sizeof(tampon), pipe) != NULL) {
23        printf("%s", tampon);
24    }
25    printf("\n-----\n");
26
27    pclose(pipe);
28 }

```

Kod 10.5: AI Soru Fonksiyonu

10.6 Komut Öneri ve Çalıştırma

AI asistanı, kullanıcının sorusuna göre komut önerir ve güvenlik kontrolü sonrası çalıştırır:

```

1 // Izin verilen komutlar (whitelist)
2 static const char* ALLOWED_COMMANDS[] = {
3     "ls", "cat", "grep", "find", "du", "df",
4     "free", "top", "ps", "uptime", "whoami",
5     "date", "cal", "pwd", "head", "tail",
6     "wc", "sort", "uniq", "awk", "cut",

```

```

7     NULL
8 };
9
10 bool is_command_safe(const char* cmd) {
11     // Tehlikeli karakterleri kontrol et
12     if (strstr(cmd, ";") || strstr(cmd, "|") ||
13         strstr(cmd, "&") || strstr(cmd, ">") ||
14         strstr(cmd, "<") || strstr(cmd, "\"") ||
15         strstr(cmd, "$")) {
16         return false;
17     }
18
19     // Whitelist kontrolu
20     for (int i = 0; ALLOWED_COMMANDS[i] != NULL; i++) {
21         if (strncmp(cmd, ALLOWED_COMMANDS[i],
22                     strlen(ALLOWED_COMMANDS[i])) == 0) {
23             return true;
24         }
25     }
26
27     return false;
28 }

```

Kod 10.6: Komut Guvenlik Kontrolu

```

1 void ai_command_helper_run(char *soru) {
2     char response[4096];
3     char *cmd_start, *cmd_end;
4
5     // AI'dan yanit al
6     // ... (ollama calistir, response'a yaz)
7
8     // "COMMAND:" satirini bul
9     cmd_start = strstr(response, "COMMAND:");
10    if (cmd_start) {
11        cmd_start += 9; // "COMMAND: " atla
12
13        // Satir sonunu bul
14        cmd_end = strchr(cmd_start, '\n');
15        if (cmd_end) *cmd_end = '\0';
16
17        // Bosluklari temizle
18        while (*cmd_start == ' ') cmd_start++;
19
20        printf("[AI] Onerilen komut: %s\n", cmd_start);
21
22        // Guvenlik kontrolu
23        if (is_command_safe(cmd_start)) {
24            printf("[AI] Komut guvenli, calistiriliyor...\n");
25            system(cmd_start);
26        } else {
27            printf("[AI] UYARI: Komut guvenlik kontrolunu gecemedi!\n");
28            printf("Komut manuel olarak calistirilamaz.\n");
29        }
30    }
31 }

```

Kod 10.7: AI Komut Yardimcisi

10.7 AI Modu Kullanım

NexOS'ta AI modunu kullanmak için ana menüden “AI OS Mode” seçilir:

```
1 [AI Mode]
2 Soru: Diskteki en büyük dosyaları bul
3 [AI] COMMAND: du -sh /* | sort -rh | head -10
4 [AI] EXPLANATION: Kök dizindeki en büyük dosya ve
5     klasörleri boyutuna göre sıralar.
6
7 Soru: Sistem bellek kullanımını göster
8 [AI] COMMAND: free -h
9 [AI] EXPLANATION: RAM ve swap kullanımını
10    okunabilir formatta gösterir.
```

Kod 10.8: AI Modu Örnekleri

Bolum 11

Entegre Web Sunucu

11.1 Genel Bakis

NexOS, BusyBox httpd tabanlı tam özellikli bir web sunucu içerir. Bu sunucu:

- Statik HTML/CSS/JS dosyaları sunar
- access.log dosyası oluşturur (log analizi için)
- Temel HTTP kimlik doğrulama (Basic Auth) destekler
- Dinamik rapor sayfalarını içerir

11.2 Web Sunucu Scripti

```
1 #!/bin/sh
2 # NexOS Public Web Server Setup
3
4 # Konfigurasyon degiskenleri
5 WEB_ROOT="${WEB_ROOT:-/var/www}"
6 LOG_DIR="${LOG_DIR:-/var/log/httpd}"
7 ACCESS_LOG="${LOG_DIR}/access.log"
8 CONFIG_FILE="${CONFIG_FILE:-/etc/httpd.conf}"
9 PORT="${PORT:-80}"
10
11 # Terminal renkleri
12 RED='\033[0;31m'
13 GREEN='\033[0;32m'
14 YELLOW='\033[1;33m'
15 NC='\033[0m'
16
17 log_info() { echo -e "${GREEN}[INFO]${NC} $1"; }
18 log_warn() { echo -e "${YELLOW}[WARN]${NC} $1"; }
19 log_error() { echo -e "${RED}[ERROR]${NC} $1"; }
```

Kod 11.1: webserver.sh - Konfigurasyon

11.3 Dizin Yapisi Kurulumu

```
1 setup_directories() {
2     log_info "Dizinler hazirlaniyor..."
3
4     # Web dizinlerini olustur
5     mkdir -p "$WEB_ROOT"/{css,js,images,reports}
6     mkdir -p "$LOG_DIR"
7
8     # Log dosyasini olustur
9     touch "$ACCESS_LOG"
10    chmod 644 "$ACCESS_LOG"
11
12    # Log dosyasini web dizinine bagla (erisim icin)
13    ln -sf "$ACCESS_LOG" "$WEB_ROOT/access.log"
14 }
```

Kod 11.2: Dizin Olusturma

11.4 Web Sayfasi Olusturma

```
1 create_website() {
2     log_info "Web sayfasi olusturuluyor..."
3
4     cat > "${WEB_ROOT}/index.html" << 'EOF'
5 <!DOCTYPE html>
6 <html>
7 <head>
8     <title>NexOS Web Server</title>
9     <style>
10         body {
11             font-family: 'Segoe UI', sans-serif;
12             background: linear-gradient(135deg, #667eea, #764ba2);
13             min-height: 100vh;
14             display: flex;
15             justify-content: center;
16             align-items: center;
17             color: white;
18         }
19         .container {
20             text-align: center;
21             padding: 40px;
22             background: rgba(255,255,255,0.1);
23             border-radius: 20px;
24             backdrop-filter: blur(10px);
25         }
26         h1 { font-size: 3em; }
27         .status {
28             background: #00ff88;
29             color: #1a1a2e;
30             padding: 10px 30px;
31             border-radius: 50px;
32             display: inline-block;
33             font-weight: bold;
34         }
```

```
35     .links a {
36         color: white;
37         text-decoration: none;
38         padding: 12px 25px;
39         margin: 5px;
40         background: rgba(255,255,255,0.2);
41         border-radius: 10px;
42         display: inline-block;
43     }
44     .links a:hover {
45         background: rgba(255,255,255,0.4);
46     }
47 </style>
48 </head>
49 <body>
50     <div class="container">
51         <h1>NexOS Server</h1>
52         <div class="status">ONLINE</div>
53         <p>Welcome to NexOS Public Web Server</p>
54
55         <div class="links">
56             <a href="/reports/">Analytics Dashboard</a>
57             <a href="/about.html">About</a>
58             <a href="/access.log">Access Log</a>
59         </div>
60     </div>
61 </body>
62 </html>
63 EOF
64 }
```

Kod 11.3: Ana Sayfa (index.html)

11.5 HTTP Kimlik Dogrulama

Hassas sayfalara erisimi kisitlamak icin Basic Authentication kullanilir:

```
1 setup_authentication() {
2     log_info "Kimlik dogrulama ayarlaniyor..."
3
4     # Admin icin sifre dosyasi olustur
5     # Format: kullanıcı:sifre
6     echo "admin:nexos123" > "${WEB_ROOT}/.htpasswd"
7     chmod 600 "${WEB_ROOT}/.htpasswd"
8
9     # httpd konfigürasyon dosyasini guncelle
10    cat >> "$CONFIG_FILE" << EOF
11 # Korunmus dizinler
12 A:admin
13 /admin
14 /reports
15 EOF
16
17    log_info "Admin kullanıcı: admin, Sifre: nexos123"
18 }
```

Kod 11.4: Kimlik Dogrulama Kurulumu

11.6 Sunucu Baslatma ve Durdurma

```
1 start_server() {
2     log_info "Web sunucu baslatiliyor..."
3
4     # Dizinleri hazirla
5     setup_directories
6     create_website
7
8     # Onceki sureci durdur (varsa)
9     pkill -f "httpd.*${WEB_ROOT}" 2>/dev/null
10
11    # BusyBox httpd'yi baslat
12    busybox httpd -p 0.0.0.0:${PORT} -h "${WEB_ROOT}" -f -vv \
13        >> "$ACCESS_LOG" 2>&1 &
14
15    # PID'i kaydet
16    echo $! > /var/run/httpd.pid
17
18    log_info "Sunucu baslatildi: http://localhost:${PORT}"
19 }
20
21 stop_server() {
22     log_info "Web sunucu durduruluyor..."
23
24     if [ -f /var/run/httpd.pid ]; then
25         kill $(cat /var/run/httpd.pid) 2>/dev/null
26         rm /var/run/httpd.pid
27         log_info "Sunucu durduruldu."
28     else
29         pkill -f "httpd.*${WEB_ROOT}" 2>/dev/null
30     fi
31 }
32
33 server_status() {
34     if pgrep -f "httpd.*${WEB_ROOT}" > /dev/null; then
35         echo -e "${GREEN}Web sunucu CALISIYOR${NC}"
36         echo "Port: ${PORT}"
37         echo "Log: $(wc -l < "$ACCESS_LOG") satir"
38     else
39         echo -e "${RED}Web sunucu DURMUS${NC}"
40     fi
41 }
```

Kod 11.5: Sunucu Kontrolu

11.7 Canli Log Takibi

```
1 tail_logs() {
2     log_info "Canli log izleme (Ctrl+C ile cik)..."
3     tail -f "$ACCESS_LOG"
4 }
```

Kod 11.6: Log Izleme

11.8 Ana Program

```
1 # Kullanım örnekleri
2 ./webserver.sh start      # Sunucuyu başlat
3 ./webserver.sh stop      # Sunucuyu durdur
4 ./webserver.sh restart   # Yeniden başlat
5 ./webserver.sh status    # Durum göster
6 ./webserver.sh logs      # Canlı log izleme
```

Kod 11.7: webserver.sh Kullanım

Bolum 12

ISO Oluşturma ve Boot Süreci

12.1 Genel Bakış

NexOS, bootable bir ISO dosyası olarak dağıtılır. Bu ISO, QEMU veya VirtualBox gibi sanal makinelerde çalıştırılır. ISO oluşturma süreci şu adımları içerir:

1. C kodlarının statik derlenmesi
2. Root dosya sisteminin hazırlanması
3. BusyBox ve Ollama'nın eklenmesi
4. Init scriptinin oluşturulması
5. Initramfs ve ISO imajının üretilmesi

12.2 Derleme Süreci

```
1 # Makefile'dan
2 CC = gcc
3 CFLAGS = -Wall -Wextra -static -O2
4
5 # Tum kaynak dosyalarini derle
6 nexos: main.o process.o memory.o scheduler.o ipc.o fs.o \
7     shell.o tui.o hardware.o utils.o ai.o
8     $(CC) $(CFLAGS) -o $@ $^
9
10 # -static: Tum kutuphaneleri icine al
11 # Sonuc: Bagimsiz calisabilen tek binary
```

Kod 12.1: Statik Derleme

12.3 Root Dosya Sistemi

```
1 rootfs/
2 +-- bin/
3 |   +-- nexos          # Ana program
4 |   +-- busybox        # Tum Unix komutlari
```

```

5 |   +-- ollama          # AI motoru
6 +-- etc/
7 |   +-- passwd         # Kullanici bilgileri
8 |   +-- udhcp.script   # DHCP yapilandirmasi
9 +-- lib/
10 |  +-- modules/       # Kernel modulleri (e1000.ko)
11 +-- root/
12 |  +-- Modelfile      # AI model tanimlama
13 |  +-- nexos-expert.gguf # AI model dosyasi
14 +-- opt/
15 |  +-- nexos/         # Ek scriptler
16 +-- var/
17 |  +-- www/           # Web sunucu dizini
18 |  +-- log/           # Log dosyalari

```

Kod 12.2: RootFS Yapisi

12.4 Init Script

Kernel başladıktan sonra ilk çalışan script:

```

1 #!/bin/sh
2
3 # 1. Temel dosya sistemlerini bagla
4 mount -t proc none /proc
5 mount -t sysfs none /sys
6 mount -t devtmpfs none /dev
7 mkdir -p /dev/pts
8 mount -t devpts devpts /dev/pts
9
10 # 2. Ortam degiskenlerini ayarla
11 export PATH=/bin:/usr/bin:/sbin
12 export HOME=/root
13 export TERM=linux
14
15 # 3. BusyBox komutlarini kur
16 busybox --install -s /bin
17
18 # 4. Karsilama mesaji
19 echo "-----"
20 echo "    NexOS Live System Booting    "
21 echo "-----"
22
23 # 5. Ag baglantisini baslat
24 echo "Starting Networking..."
25
26 # VirtualBox/QEMU ag suruculerini yukle
27 if [ -f /lib/modules/e1000.ko ]; then
28     insmod /lib/modules/e1000.ko
29 fi
30
31 # Ag arayuzunu bul ve DHCP ile yapilandir
32 for iface in $(ls /sys/class/net/ | grep -v lo); do
33     ip link set $iface up
34     udhcpd -b -i $iface -s /etc/udhcp.script &
35 done
36

```

```

37 # 6. Ollama AI servisini baslat
38 echo "Starting AI Engine..."
39 mkdir -p /root/.ollama
40 nohup /bin/ollama serve > /var/log/ollama.log 2>&1 &
41
42 # Ollama'nin hazir olmasini bekle
43 for i in 1 2 3 4 5; do
44     sleep 2
45     if wget -q -O /dev/null http://127.0.0.1:11434/api/tags; then
46         echo "Ollama hazir!"
47         break
48     fi
49 done
50
51 # 7. AI modelini olustur
52 if [ -f /root/Modelfile ]; then
53     /bin/ollama create nexos-expert -f /root/Modelfile
54 fi
55
56 # 8. Ana programi sonsuz dongude calistir
57 echo "Starting NexOS..."
58 while true; do
59     /bin/nexos
60     echo "NexOS kapandi. Yeniden baslatiliyor..."
61     sleep 2
62 done

```

Kod 12.3: /init - Boot Scripti

12.5 Initramfs Oluşturma

```

1 # rootfs dizininden initramfs olustur
2 cd rootfs
3 find . | cpio -H newc -o | gzip > ../initramfs.cpio.gz
4
5 # Sonuc: ~50-100MB sikistirilmis dosya sistemi

```

Kod 12.4: Initramfs Paketleme

12.6 ISO İmajı Oluşturma

```

1 # ISO dizin yapisi
2 iso/
3 +-- boot/
4 |   +-- isolinux/
5 |   |   +-- isolinux.bin      # Bootloader
6 |   |   +-- isolinux.cfg      # Boot konfigurasyonu
7 |   |   +-- ldlinux.c32       # Gerekli modul
8 |   +-- vmlinuz               # Linux kernel
9 |   +-- initramfs.cpio.gz      # Root dosya sistemi
10 +-- NexOS
11
12 # Boot konfigurasyonu (isolinux.cfg)
13 DEFAULT nexos

```

```
14 LABEL nexos
15     KERNEL /boot/vmlinuz
16     APPEND initrd=/boot/initramfs.cpio.gz
17
18 # ISO olusturma komutu
19 xorriso -as mkisofs \
20     -o nexos.iso \
21     -b boot/isolinux/isolinux.bin \
22     -c boot/isolinux/boot.cat \
23     -no-emul-boot \
24     -boot-load-size 4 \
25     -boot-info-table \
26     -isohybrid-mbr /usr/lib/ISOLINUX/isohdpx.bin \
27     iso/
```

Kod 12.5: ISO Olusturma

12.7 ISO Çalıştırma

```
1 # Temel calistirma
2 qemu-system-x86_64 -cdrom nexos.iso -m 2G
3
4 # KVM hizlandirmasi ile (Linux)
5 qemu-system-x86_64 -cdrom nexos.iso -m 2G -enable-kvm
6
7 # Ag ile
8 qemu-system-x86_64 -cdrom nexos.iso -m 2G \
9     -nic user,hostfwd=tcp::8080-:80
```

Kod 12.6: QEMU ile Test

Bolum 13

Kullanıcı Arayüzü

13.1 Giriş Ekranı

NexOS başladığında kullanıcı adı ve şifre istenir:

- Varsayılan kullanıcı: `admin`
- Varsayılan şifre: `123`

13.2 Ana Menü

Giriş başarılı olunca TUI menüsü gösterilir:

1. AI OS Mode - Yapay zeka destekli mod
2. Manual Mode - Uzman kabuğu
3. System Information - Sistem bilgisi
4. Web Server Control - Web sunucu yönetimi
5. Tools - Araçlar
6. Power Off - Kapat

13.3 Shell Komutları

Komut	Açıklama
<code>help</code>	Tüm komutları listele
<code>proclist</code>	Süreç tablosunu göster
<code>newproc <ad></code>	Yeni süreç oluştur
<code>memdemo</code>	Bellek izolasyonu demo
<code>scheddemo</code>	Zamanlayıcı demo
<code>ipcdemo</code>	IPC demo
<code>analyze <dosya></code>	Log dosyası analizi
<code>traffic</code>	Trafik analizi
<code>web</code>	Web sunucu başlat
<code>exit</code>	Çıkış

Bolum 14

Sonuç

14.1 Özellik Özeti

NexOS aşağıdaki işletim sistemi kavramlarını içerir:

- **Süreç Yönetimi:** Oluşturma, sonlandırma, durum geçişleri
- **Bellek İzolasyonu:** Her süreç için güvenli ayrı alan
- **CPU Zamanlama:** Round-Robin algoritması
- **IPC:** Mesaj tabanlı süreçler arası iletişim
- **Dosya Sistemi:** Kalıcı sanal disk
- **Log Analizi:** awk/sort/uniq ile modüler shell programlama
- **AI Entegrasyonu:** Ollama tabanlı komut asistanı
- **Web Sunucu:** BusyBox httpd ile HTTP sunumu
- **Boot Süreci:** Initramfs ve ISO oluşturma

14.2 Öğrenilen Kavramlar

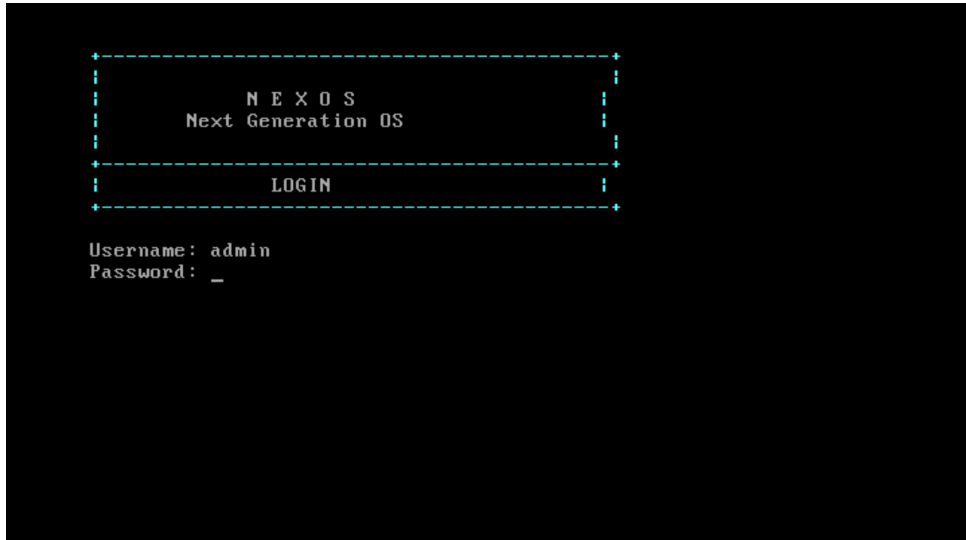
1. C ile sistem programlama
2. Süreç ve bellek yönetimi
3. Zamanlama algoritmaları
4. Süreçler arası iletişim
5. Log dosyası analizi
6. Bootable ISO oluşturma

Bolum 15

Ekler

15.1 Ekran Görüntüleri

15.1.1 Giriş Ekranı



15.1.2 Ana Ekran



15.1.3 Yardım Sayfası

```

+-----+
|  HELP MENU  |
+-----+
| --- COMMAND LIST --- |
| analyze <file> : Log file analysis |
| sysinfo : Real hardware report |
| ls : View virtual files (SFS) |
| write <n> <t> : Create virtual file |
| ai : Ask questions to AI |
| lss : List system files (ls -l) |
| cat <file> : Read file content |
| less <file> : Read file (Scrollable) |
| ps : List processes |
| kill <PID> : Kill process |
| ping <addr> : Internet test |
| free : Memory status |
| clean : Clear cache (Free RAM) |
| web : Start Web Server (Port 80) |
| vi <file> : Text Editor |
| top : Process Monitor (Exit: q) |
| wget <url> : Download File |
+-----+
| [Up/Down] Scroll | [PgUp/Dn] Page | [Q] Quit | Lines 1-18 of 34 |
+-----+

```

15.1.4 Süreç Yönetimi (proc kill)

```

root@nexos:/$ proclist
+-----+
|          PROCESS TABLE (Süreç Tablosu)          |
+-----+
| PID | Name | State | Prio | Memory |
+-----+
| 1000 | yeni | READY | 5 | 0/4096 bytes |
+-----+
Total: 1 active processes
root@nexos:/$ killproc 1000
[PROCESS] Terminated: yeni (PID: 1000)
root@nexos:/$ proclist
+-----+
|          PROCESS TABLE (Süreç Tablosu)          |
+-----+
| PID | Name | State | Prio | Memory |
+-----+
|      |      |      |      |      |
+-----+
|      |      |      |      |      |
+-----+
(No active processes)
+-----+
Total: 0 active processes

```

15.1.5 Sistem İzleme (monitor)

```

--- TOP Paths ---
1 /reports/
1 /login
1 /api/status
1 /about.html
1 /

--- TOP Errors ---
1 404

--- SECURITY ALERTS ---
[OK] No common attack signatures found.
[WARNING] Access to Sensitive Paths:
10.0.2.2:56058: url:/login

--- USER JOURNEYS (Last 10 Actions) ---
[10.0.2.2:56050] VISITED /
[10.0.2.2:56051] VISITED /reports/
[10.0.2.2:56057] VISITED /about.html
[10.0.2.2:56058] VISITED /login
[10.0.2.2:56059] VISITED /api/status

```

15.1.6 Log Analiz Çıktısı

```

1 10.0.2.2:56058
1 10.0.2.2:56057
1 10.0.2.2:56051
1 10.0.2.2:56050

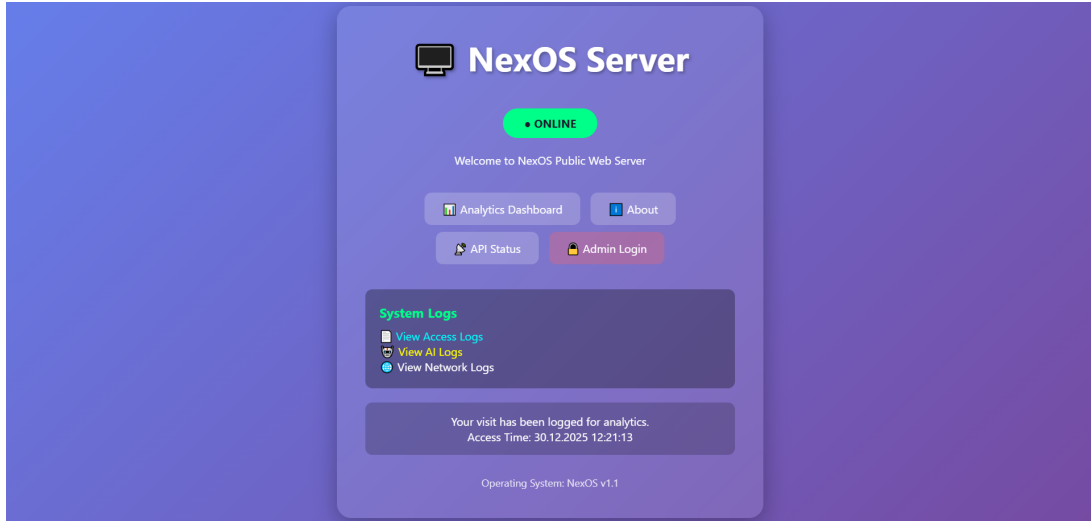
--- TOP Paths ---
1 /reports/
1 /login
1 /api/status
1 /about.html
1 /

--- TOP Errors ---
1 404

--- SECURITY ALERTS ---
[OK] No common attack signatures found.
[WARNING] Access to Sensitive Paths:
10.0.2.2:56058: url:/login

```

15.1.7 Web Sitesi Görünümü



15.1.8 Yapay Zeka Yanıtı

```

AI OS ['exit' for menu]> list all the files
[All NexOS Expert Thinking...

+-----+
| AI Response |
+-----+
Explanation: Lists all files recursively.

Command: ls
Execute? [Y/n]: Y
Executing...
access.log  etc          lib64        root         tmp
bin         init         opt          sbin         usr
dev         lib          proc         sys          var

```